

# Modeling NoSQL databases

Modeling collections

**Adapted from:**

MongoDB: <https://learn.mongodb.com/>

Philippe Rigaux: <http://www.bdpedia.fr/>

# Modeling NoSQL databases

In the context of the implementation of a large database consisting of documents:

- **How to model these documents to satisfy the needs of the application?**
- **What is the structure of these documents?**
- **What are the constraints on the content of the documents?**

This question **is well known** in the context of **relational databases (SQL)**, and we will start by recalling the well-established method.

For **NoSQL databases**, **there is no equivalent methodology**. A good (or bad) reason is that there is **no standardized model**, and that the **modeling must adapt to the characteristics of each system**.

**We shall extrapolate relational design methodology to study what can be achieved with a structured document model.**

# Modeling a relational database (reminder)

## Entity/Relations (ER) - Essentially:

- Determine the "entities" (movie, directors, actors) relevant to the application;
- Define a way for identifying each entity;
- Preserve the link between the entities.

Example for a "Movies" application:

- There are two types of entities (to start with): **movies and artists**.
- They are identified by an incremental number (the **id**).
- In each **movie**, the artist is referenced by his/her **identifier**.

# Illustration

**Movies table**

| id | title        | year |
|----|--------------|------|
| 1  | Alien        | 1979 |
| 2  | Vertigo      | 1958 |
| 3  | Psychose     | 1960 |
| 4  | Kagemusha    | 1980 |
| 5  | Volte-face   | 1997 |
| 6  | Pulp Fiction | 1994 |
| 7  | Titanic      | 1997 |
| 8  | Sacrifice    | 1986 |

**Artists table**

| id  | surname   | name    | DOB  |
|-----|-----------|---------|------|
| 101 | Scott     | Ridley  | 1943 |
| 102 | Hitchcock | Alfred  | 1899 |
| 103 | Kurosawa  | Akira   | 1910 |
| 104 | Woo       | John    | 1946 |
| 105 | Tarantino | Quentin | 1963 |
| 106 | Cameron   | James   | 1954 |
| 107 | Tarkovski | Andrei  | 1932 |

**Can you tell which movie Scott Ridley played in?**

**A link between Movies and Artists is missing!**

# The relational referencing system

**Movies table**

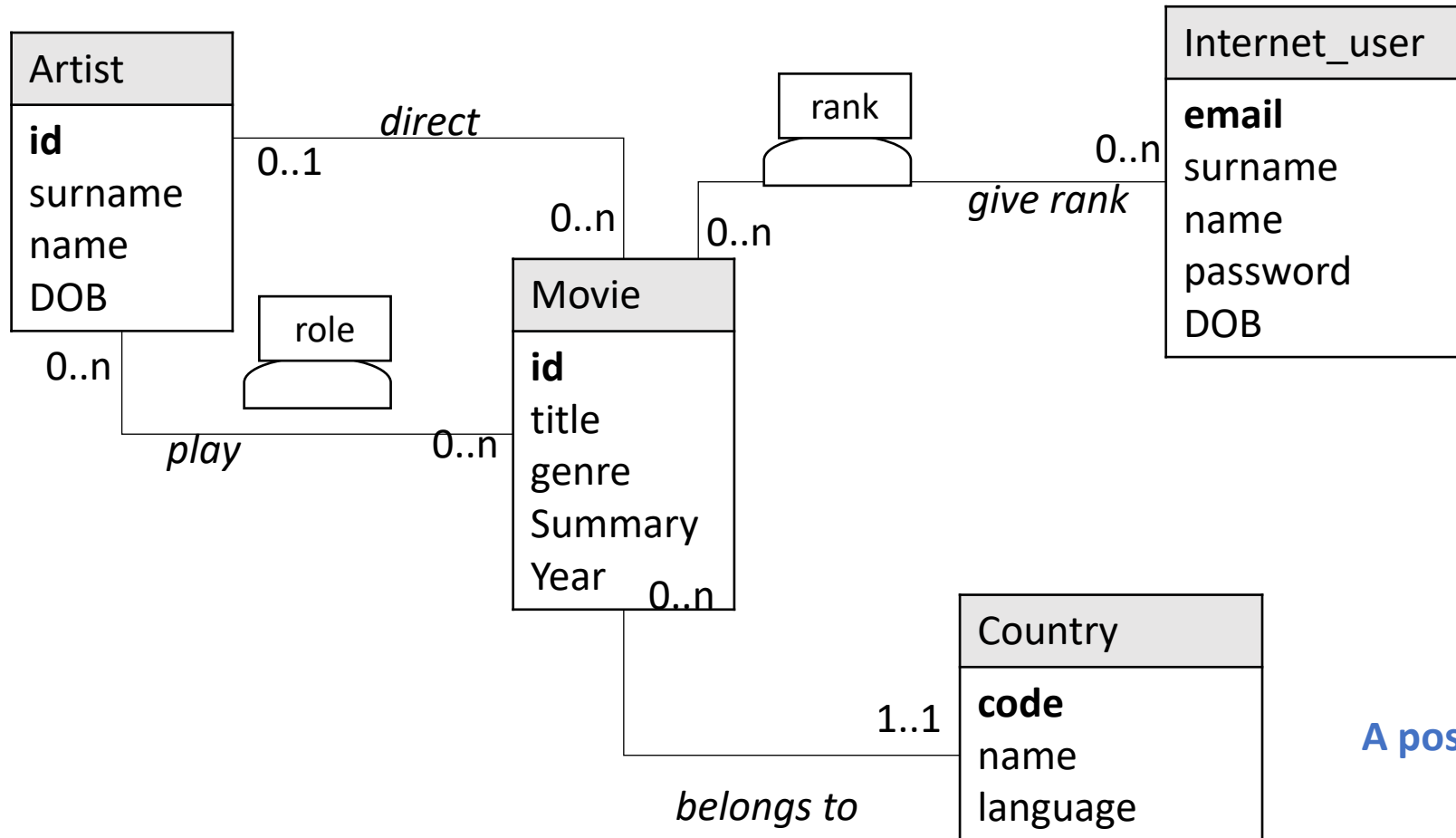
| id | title        | year | idDirector |
|----|--------------|------|------------|
| 1  | Alien        | 1979 | 101        |
| 2  | Vertigo      | 1958 | 102        |
| 3  | Psychose     | 1960 | 102        |
| 4  | Kagemusha    | 1980 | 103        |
| 5  | Volte-face   | 1997 | 104        |
| 6  | Pulp Fiction | 1994 | 105        |
| 7  | Titanic      | 1997 | 106        |
| 8  | Sacrifice    | 1986 | 107        |

**Artists table**

| id  | surname   | name    | DOB  |
|-----|-----------|---------|------|
| 101 | Scott     | Ridley  | 1943 |
| 102 | Hitchcock | Alfred  | 1899 |
| 103 | Kurosawa  | Akira   | 1910 |
| 104 | Woo       | John    | 1946 |
| 105 | Tarantino | Quentin | 1963 |
| 106 | Cameron   | James   | 1954 |
| 107 | Tarkovski | Andrei  | 1932 |

**It is not a physical link, but it allows joins!**

# Entity/relation model



A possible model for the movie database

# Relational schema (SQL)

```
create table Artist (idArtist integer not null,  
                    surname varchar (30) not null,  
                    name varchar (30) not null,  
                    DOB Date,  
                    primary key (idArtist),  
                    unique (surname, name));
```

| Artist    |
|-----------|
| <b>id</b> |
| surname   |
| name      |
| DOB       |

The table Artist in the Movie database

Expresses **constraints**: primary key, uniqueness (surname, name), types...

# Relational schema (SQL)

```
create table Movie (idMovie integer not null,  
                    title    varchar (50) not null,  
                    year     integer not null,  
                    idDirector integer not null,  
                    genre    varchar (20) not null,  
                    summary   varchar(255),  
                    CountryCode varchar (4),  
                    primary key (idMovie),  
                    foreign key (idDirector) references Artist);
```

| Movie   |
|---------|
| id      |
| title   |
| genre   |
| summary |

The **foreign key** referential integrity constraint guarantees the **consistency** of the referencing.



# Normalization

In relational, all data is "flat".

This tends to multiply the number of tables containing information on the same entity.

**Example:** to represent all the information on a movie, you need:

- The **Movie** table (one line for the movie)
- The **Artist** table (one row for the director, as many rows as there are actors)
- The **Role** table (as many rows as roles)
- and the **Country** table, and more...

It takes a lot of **writing**, and as many **readings** to reconstruct the information.

# Relational representation of the movie ('Pulp fiction')

**Movies:**

| id | title        | year | idDirector |
|----|--------------|------|------------|
| 6  | Pulp Fiction | 1994 | 37         |

**Artists:**

| id | surname   | name    | DOB  |
|----|-----------|---------|------|
| 11 | Travolta  | John    | 1954 |
| 27 | Willis    | Bruce   | 1955 |
| 37 | Tarantino | Quentin | 1963 |

**Roles:**

| idMovie | idArtist | role           |
|---------|----------|----------------|
| 6       | 11       | Vincent Vega   |
| 6       | 27       | Butch Coolidge |
| 6       | 37       | Jimmy Dimmick  |

# Impact of normalization

- **Several entries** must be made for the same entity (transactions)
- It is necessary to perform **joins** to reconstitute an entity.

Reconstitution of information on a movie in SQL:

```
select title, surname, name, role
from Movie , Artist , Role
where role='Vincent Vega'
and Movie.id = Role.idMovie
and Artist.id = Role.idArtist
```

# Power of the structured document model

- Structured documents are not subject to normalization constraints.
- An attribute can have **multiple values** (using array structure).

```
{  
  "title": "Pulp Fiction",  
  "year": "1994",  
  "genre": ["Action", "Crime", "Comedy"]  
  "country": "USA"  
}
```

- In relational, it would be necessary to add two tables.

# Power of the structured document model (2)

- It is also easy to represent regular data.

| id  | surname   | name    |
|-----|-----------|---------|
| 37  | Tarantino | Quentin |
| 167 | De Niro   | Robert  |
| 168 | Grier     | Pam     |

- Easy to represent in the form of a document (very regular).

```
{  
  "artist": [{  
    "id": 37, "surname": "Tarantino", "name": "Quentin"},  
    {"id": 167, "surname": "De Niro", "name": "Robert"},  
    {"id": 168, "surname": "Grier ", "name": "Pam"}]  
}
```

Relational table = tree of constants. Three levels: line, attribute, value.

# More powerful than relational representation?

- Can be useful for **exchanging** data; not to **store** it.
- In a relational database, the data is very **constrained/structured**: we can store **structure (schema)** and **content (base)** separately.
- An XML / JSON tree representation is more appropriate for data of **complex** and/or **flexible** structure.

# Structured documents = nesting of structures

- Thanks to **the nesting of the structures**, it is possible to represent in the same unity a movie **and** its **director**.

```
{ "title": "Pulp fiction",  
  "year": "1994",  
  "genre": "Action",  
  "country": "USA",  
  "director": { "surname": "Tarantino",  
                "name": "Quentin",  
                "birth_date": "1963"  
              }  
}
```

- In document databases: we try to create **autonomous** information units to limit entries and avoid having to make joins.

NoSQL systems are designed to scale by distribution. This is partly incompatible with joins and transactions.

# Representation as a structured document

- We can encode all the information on a movie.

```
{
  "_id": "movie:17",
  "title": "Pulp Fiction",
  "year": "1994",
  "director": {
    "last_name": "Tarantino", "first_name": "Quentin",
    "birth_date": "1963"
  },
  "actors": [
    {
      "first_name": "John", "last_name": "Travolta",
      "birth_date": "1954", "role": "Vindent Vega"
    },
    {
      "first_name": "Bruce", "last_name": "Willis",
      "birth_date": "1955", "role": "Butch Coolidge"
    },
    {
      "first_name": "Quentin", "last_name": "Tarantino",
      "birth_date": "1963", "role": "Jimmy Dimmick"
    }
  ]
}
```



# Summary to remember

The advantages of structured document representation:

- **No more need for joins (?)**: it is useless to make joins to reconstitute information since it is no longer dispersed, as in relational, in several tables.
- **No more need for a transaction (?)**: one writing (of the document) is sufficient; one lecture enough to retrieve all the information.
- **Adaptation to distribution**. If the documents are self-contained, it is very easy move them to best distribute them in a distributed system.

A NoSQL system is much easier to implement than a relational system. A *put()* operation, a *get()* operation and off you go

# The disadvantages of the document model

- The document model has two major drawbacks:
- **Preferred Path:** Movies appear near the **root** of the documents; the artists are buried in the depths; Access to movies is therefore privileged.
- **Entities are no longer autonomous:** no way to create a **director** if there **isn't at least one movie**.
- **Redundancy:** the same information must be represented several times, which is quite annoying (Quentin Tarantino is represented twice).

Redundancy leads to inconsistencies. Preferring a path is good for some applications, bad for others.

# Other disadvantages

- No schema!

You can put anything in the base; it is the application that must do the checks and corrections.

No transactions!

Not suitable for many applications (e-commerce).

# NoSQL or relational(SQL)?

---

## Consider SQL databases when...

- Your data is highly structured, and that structure doesn't change frequently
- You support transaction-oriented systems such as accounting or financial applications
- You require a high degree of data integrity and security
- You routinely perform complex queries, including ad hoc requests
- You don't require the scale-out capabilities that NoSQL offers

## Consider NoSQL databases when...

- You're working with large amounts of unstructured or semi-structured data that doesn't fit the relational model
  - You require the flexibility of a dynamic schema or want more choice over the data model
  - You require a database system that can be scaled horizontally, perhaps across multiple geographic locations
  - You want to streamline development and avoid the overhead of a more structured approach
  - Your applications don't require the level of data integrity offered by SQL databases
-

# NoSQL or relational (SQL)

Always ask yourself (at least) the following questions:

- Is it a question of representing a regular structure (e.g. a table)?

In this case it is not necessary to integrate the structure and the content.

Do I often have to modify existing data?

In this case I probably need transactions, and I risk paying dearly for any redundancy.

Is the structure of a document fixed and predictable?

If so, the relational model is well suited.

NoSQL = Not Only SQL. Nobody (seriously) claims that these systems will replace relational systems, except for certain niches!



SQL(relational) vs. NoSQL, wedding cake style